

Deploy Inventory on Ubuntu 24.04

VPS deployment guide | Laravel 12 + Next.js + Expo client

Use **Nginx**, **PHP 8.3-FPM**, **MySQL** and **Node.js 24**, with systemd keeping the website and queue worker running. These instructions assume a fresh installation, a sudo-capable Ubuntu account and a domain pointing to your VPS.

Replace **inventory.example.com**, **YOUR_REPOSITORY_URL** and password placeholders throughout.

Directory	Deployment role
web/	Next.js website
inventory-api/	Laravel API
mobile/	Expo app; connects to the deployed API
salepro-clone/	Reference code; no deployment needed

```
https://inventory.example.com      -> Next.js
https://inventory.example.com/api/* -> Laravel
https://inventory.example.com/storage/* -> Uploaded files
```

1. Install server dependencies

Run on the VPS:

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y \
    nginx mysql-server git curl unzip acl cron \
    php8.3-cli php8.3-fpm php8.3-mysql php8.3-sqlite3 \
    php8.3-mbstring php8.3-xml php8.3-curl php8.3-zip \
    php8.3-bcmath php8.3-intl php8.3-gd \
    composer
```

Install Node.js 24 using the NodeSource repository:

```
curl -fsSL https://deb.nodesource.com/setup_24.x \
    -o /tmp/nodesource_setup.sh
sudo bash /tmp/nodesource_setup.sh
sudo apt install -y nodejs
php -v
node -v
composer --version
sudo systemctl enable --now nginx mysql php8.3-fpm cron
```

Database and application configuration

2. Create a deployment user and download the project

```
sudo adduser deploy
sudo usermod -aG sudo deploy
sudo mkdir -p /var/www/inventory
sudo chown deploy:deploy /var/www/inventory
sudo -iu deploy
git clone YOUR_REPOSITORY_URL /var/www/inventory
```

Run all remaining commands as **deploy**, using `sudo` where shown.

3. Create the database

```
sudo mysql
```

Run this SQL, replacing the password:

```
CREATE DATABASE inventory
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
CREATE USER 'inventory'@'localhost'
  IDENTIFIED BY 'REPLACE_WITH_A_LONG_RANDOM_PASSWORD';
GRANT ALL PRIVILEGES ON inventory.*
  TO 'inventory'@'localhost';
EXIT;
```

4. Configure Laravel

```
cd /var/www/inventory/inventory-api
cp .env.example .env
nano .env
```

Set these values, preserving the other settings:

```
APP_NAME="Inventory"
APP_ENV=production
APP_DEBUG=false
APP_URL=https://inventory.example.com
LOG_LEVEL=warning
DB_CONNECTION=mysql
DB_HOST=localhost
DB_PORT=3306
DB_DATABASE=inventory
DB_USERNAME=inventory
DB_PASSWORD="REPLACE_WITH_A_LONG_RANDOM_PASSWORD"
SESSION_DRIVER=database
SESSION_SECURE_COOKIE=true
CACHE_STORE=database
QUEUE_CONNECTION=database
MAIL_MAILER=log
```

The example environment file contains Mailtrap settings. Replace them with your own SMTP configuration when enabling email. **MAIL_MAILER=log records messages locally; it does not deliver them.**

Initialize Laravel

4. Configure Laravel (continued)

Install production dependencies and generate the application key:

```
cd /var/www/inventory/inventory-api
composer install --no-dev --prefer-dist --optimize-autoloader
composer check-platform-reqs --no-dev
php artisan key:generate
```

Give PHP and the deployment user access to Laravel's writable directories:

```
sudo chgrp www-data .env
chmod 640 .env
sudo setfacl -R \
  -m u:deploy:rwX,u:www-data:rwX \
  storage bootstrap/cache
sudo setfacl -dR \
  -m u:deploy:rwX,u:www-data:rwX \
  storage bootstrap/cache
```

For a **new database**:

```
php artisan migrate --force
php artisan db:seed --force
php artisan storage:link
```

Project-specific detail: the seeder creates roles, settings, sample business records and admin@example.com with password **password**. Change that account before exposing the site.

```
php artisan tinker
```

Inside Tinker:

```
$user = App\Models\User::where('email', 'admin@example.com')->firstOrFail();

$user->forceFill([
    'email' => 'you@example.com',
    'password' => Illuminate\Support\Facades\Hash::make(
        \Laravel\Prompts\password('New admin password')
    ),
])->save();

exit
```

Then cache the production configuration:

```
php artisan optimize
```

Moving existing inventory data? Restore your database and uploaded files instead of seeding. Preserve the existing APP_KEY rather than generating a new one.

Laravel requires writable storage and bootstrap/cache directories and recommends production configuration caching. See the Laravel deployment reference on the final page.

Run the website and background jobs

5. Build the Next.js website

```
cd /var/www/inventory/web
cat > .env.production.local <<'EOF'
NEXT_PUBLIC_API_URL=https://inventory.example.com/api
EOF
npm ci
npm run build
```

Set the API URL **before building**. Next.js embeds NEXT_PUBLIC_* values in the browser bundle; changing them requires another build.

Create /etc/systemd/system/inventory-web.service with sudo nano:

```
[Unit]
Description=Inventory Next.js website
After=network.target

[Service]
Type=simple
User=deploy
Group=deploy
WorkingDirectory=/var/www/inventory/web
Environment=NODE_ENV=production
ExecStart=/usr/bin/node /var/www/inventory/web/node_modules/next/dist/bin/next \
  start --hostname 127.0.0.1 --port 3000
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

6. Configure the queue worker and scheduler

Your API uses queued invoice emails and schedules sales-invoice-emails:recover every minute. **Both components need to run.**

Create /etc/systemd/system/inventory-queue.service with sudo nano:

```
[Unit]
Description=Inventory Laravel queue worker
After=network.target mysql.service

[Service]
Type=simple
User=www-data
Group=www-data
WorkingDirectory=/var/www/inventory/inventory-api
ExecStart=/usr/bin/php artisan queue:work --sleep=3 --tries=3 --timeout=60
Restart=always
RestartSec=5
KillSignal=SIGTERM
TimeoutStopSec=120

[Install]
WantedBy=multi-user.target
```

Scheduler and Nginx routing

6. Start services and enable the scheduler

```
sudo systemctl daemon-reload
sudo systemctl enable --now inventory-web inventory-queue
sudo nano /etc/cron.d/inventory
```

Paste the following into the cron file, then save with a final newline:

```
INVENTORY_API=/var/www/inventory/inventory-api
* * * * * www-data cd $INVENTORY_API && /usr/bin/php artisan schedule:run >> storage/logs/scheduler.log 2>&1

sudo chmod 644 /etc/cron.d/inventory
```

Long-running queue workers must restart when application code changes.

7. Configure Nginx

```
sudo nano /etc/nginx/sites-available/inventory
```

Paste this configuration. Only Laravel's front controller may execute PHP:

```
server {
    listen 80;
    server_name inventory.example.com;
    root /var/www/inventory/inventory-api/public;
    client_max_body_size 20M;
    location = /api { rewrite ^ /index.php last; }
    location ^~ /api/ { rewrite ^ /index.php last; }
    location = /up { rewrite ^ /index.php last; }
    location /storage/ { try_files $uri =404; }

    location = /index.php {
        internal;
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $document_root/index.php;
        fastcgi_param HTTP_AUTHORIZATION $http_authorization;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
    }
    location ~ \.php(?:/|$) { return 404; }
    location ~ /\.(!well-known).* { deny all; }

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_buffering off;
    }
}
```

Enable routing and HTTPS

7. Activate Nginx and configure uploads

```
sudo ln -s /etc/nginx/sites-available/inventory \
    /etc/nginx/sites-enabled/inventory
sudo nginx -t
sudo systemctl reload nginx
```

Match PHP's upload limits to Nginx:

```
sudo tee /etc/php/8.3/fpm/conf.d/99-inventory.ini >/dev/null <<'EOF'
upload_max_filesize = 20M
post_max_size = 24M
EOF
sudo systemctl restart php8.3-fpm
```

8. Enable the firewall and HTTPS

Ensure the domain's DNS A record points to your VPS. Any AAAA record must point to working IPv6 on this server.

For SSH on the standard port:

```
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw enable
```

If SSH uses a custom port, allow that port before enabling UFW. Also allow TCP 80 and 443 in your VPS provider's firewall.

Install Certbot and request HTTPS:

```
sudo apt install -y snapd
sudo snap install --classic certbot
sudo /snap/bin/certbot --nginx \
    -d inventory.example.com --redirect
sudo /snap/bin/certbot renew --dry-run
```

Certbot updates the Nginx configuration and sets up automatic renewal.

9. Verify the deployment

```
sudo systemctl status inventory-web inventory-queue --no-pager
curl -I https://inventory.example.com
curl -i https://inventory.example.com/up
curl -i \
    -H 'Accept: application/json' \
    https://inventory.example.com/api/me
```

Expected: the website returns a successful response or redirect; /up returns HTTP 200; /api/me returns HTTP 401 without a login token.

Validation, mobile access and updates

9. Verify the deployment (continued)

Sign in and test product creation, uploads and invoice generation. Configure real SMTP and notification settings before testing email delivery.

Useful logs:

```
sudo journalctl -u inventory-web -n 100 --no-pager
sudo journalctl -u inventory-queue -n 100 --no-pager
tail -n 100 /var/www/inventory/inventory-api/storage/logs/laravel.log
sudo tail -n 100 /var/log/nginx/error.log
```

10. Connect the mobile app

In the mobile app's build environment:

```
EXPO_PUBLIC_API_URL=https://inventory.example.com/api
```

Rebuild the mobile app to include the production URL.

Future deployments

For a simple update with a maintenance window:

1. Back up MySQL, inventory-api/storage/app and the production .env.
2. Stop inventory-web and inventory-queue, then run php artisan down.
3. Pull the updated code.
4. In inventory-api/, run Composer install, php artisan migrate --force and php artisan optimize.
5. In web/, run npm ci and npm run build.
6. Start both services and run php artisan up from inventory-api/.

Do not rerun the main seeder during routine updates: this project's seeders overwrite some business settings and sample records. Preserve your production .env, APP_KEY and uploads.

Official references

[Next.js self-hosting](#)

[NodeSource installation instructions](#)

[Laravel 12 deployment](#)

[Laravel 12 queues](#)

[Certbot instructions for Nginx](#)

Prepared from the repository inspection and the deployment guide in this conversation. These instructions have not been deployed or tested on your VPS. Exported 15 September 2026.